CineBook

A Movie Ticket Booking System Using Oracle PL/SQL

Mini Project Report

BCSE302P – Database Systems Laboratory

Submitted by

Ishaan Singh (24BCE0671)

Under the guidance of

Dr. G. M. Karthik

School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Date of submission: 24 October 2026

Certificate

I hereby declare that the mini project report entitled “CineBook: A Movie Ticket Booking System Using Oracle PL/SQL”, submitted for BCSE302P – Database Systems Laboratory at Vellore Institute of Technology, Vellore, is a record of original work carried out by me under the guidance of Dr. G. M. Karthik. All code and screenshots in this report are from my own execution, and this work has not been submitted in whole or in part for any other course or degree.

Ishaan Singh
Registration number: 24BCE0671
Place: Vellore
Date: 24 October 2026

Abstract

CineBook is a database-driven movie ticket booking system built on Oracle AI Database 26ai Free. It models a two-screen cinema with users, movies, screens, individual seats, shows, bookings, booked seats and an audit log, stored in eight relations normalised to Boyce–Codd Normal Form. The business logic lives in the database: a PL/SQL package, movie_pkg, provides four functions (seat pricing, seat availability, refund calculation and a seat map returned as a REF CURSOR) and four procedures (booking, cancellation, a per-show report and a daily schedule). Three triggers price each booked seat, keep booking totals up to date and log every cancellation with its refund. A booking locks its show row with SELECT ... FOR UPDATE, so two customers can never reserve the same seat, and any failure rolls the whole booking back. Sixteen scripted test cases cover normal bookings, premium pricing, double booking, invalid input, the seat limit, refund rules and constraint violations, and all sixteen pass. A desktop web front-end, served by a Python FastAPI server through python-oracledb, lets users browse showtimes, pick seats on a live seat map, view and cancel tickets, and run the PL/SQL reports. Online payment and password authentication are outside the project's scope.

Table of Contents

1. Introduction..... 5

2. System Requirements..... 5

3. Database Design..... 6

4. Implementation..... 7

5. Front-End..... 12

6. Verification & Testing..... 13

7. Results & Discussion..... 14

8. Conclusion..... 15

References..... 15

Appendices..... 16

1. Introduction

1.1 Problem statement

Small cinemas often sell tickets from a paper seat chart or a spreadsheet. The same seat can be sold twice, refunds depend on who is at the counter, and there is no reliable record of sales and cancellations. CineBook is a movie ticket booking system that keeps all of this in an Oracle database, so seat inventory, pricing, refunds and history are always correct.

1.2 Objectives

- Design a normalised schema with integrity constraints for movies, screens, seats, shows, users and bookings.
- Implement the business rules in PL/SQL using functions, procedures, a package, triggers and cursors.
- Prevent double booking of a seat, even when two bookings arrive at the same time.
- Calculate seat prices and refunds automatically and log every booking and cancellation.
- Verify every rule with test cases and connect a front-end to the database.

1.3 Importance of DBMS

A DBMS is essential for this problem. Row-level locking lets many users book at once without conflicts; transactions make a multi-seat booking all-or-nothing; constraints reject invalid data from any program; and PL/SQL keeps the rules next to the data, so every client, whether SQL*Plus or the application, behaves the same way.

2. System Requirements

Table 1: Hardware and software requirements

Category	Requirement
Processor	64-bit Intel or AMD, dual-core or better
Memory and storage	8 GB RAM, about 10 GB free disk space
Operating system	Windows 11
DBMS	Oracle AI Database 26ai Free, Release 23.26.2 (pluggable database FREEPDB1)
Database tool	SQL*Plus 23.26.2
Language and libraries	Python 3.14, FastAPI, Uvicorn, python-oracledb (thin mode)
Front-end	HTML, CSS and JavaScript, run in its own window through Microsoft Edge app mode

3. Database Design

3.1 ER diagram

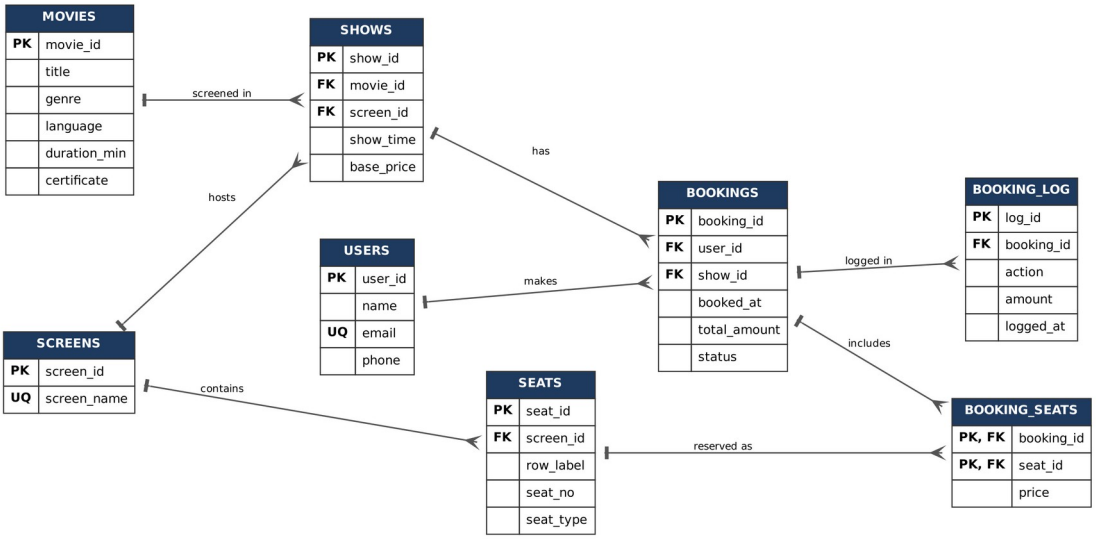


Figure 1: ER diagram of CineBook (crow's-foot notation)

A movie has many shows; a screen has many shows and many seats; a user makes many bookings; a show has many bookings (all 1 : N). A booking includes many seats and a seat is booked many times across shows (M : N), resolved by BOOKING_SEATS, which also stores the price charged. BOOKING_LOG records every booking and cancellation.

3.2 Schema definitions with constraints

Table 2: Schema of the eight tables

Table	Columns	Constraints
USERS	user_id (PK), name, email (UQ), phone	CHECK phone is 10 digits starting 6–9
MOVIES	movie_id (PK), title, genre, language, duration_min, certificate	CHECK duration > 0; certificate IN (U, UA, A)
SCREENS	screen_id (PK), screen_name (UQ)	
SEATS	seat_id (PK), screen_id (FK), row_label, seat_no, seat_type	UQ (screen_id, row_label, seat_no); type REGULAR or PREMIUM
SHOWS	show_id (PK), movie_id (FK), screen_id (FK), show_time, base_price	UQ (screen_id, show_time); price > 0
BOOKINGS	booking_id (PK), user_id (FK), show_id (FK), booked_at, total_amount, status	status CONFIRMED or CANCELLED; total set by trigger
BOOKING_SEATS	booking_id (PK, FK), seat_id (PK, FK), price	price set by trigger
BOOKING_LOG	log_id (PK), booking_id (FK), action, amount, logged_at	action BOOKED or CANCELLED

All 27 constraints are named so that errors identify the rule broken, for example CHK_USERS_PHONE. The BOOKINGS table shows the style used:

```
CREATE TABLE bookings (
  booking_id   NUMBER GENERATED ALWAYS AS IDENTITY,
  user_id      NUMBER NOT NULL,
  show_id      NUMBER NOT NULL,
  booked_at    TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
  total_amount NUMBER(8,2) DEFAULT 0 NOT NULL,
  status       VARCHAR2(10) DEFAULT 'CONFIRMED' NOT NULL,
  CONSTRAINT pk_bookings PRIMARY KEY (booking_id),
  CONSTRAINT fk_bookings_user FOREIGN KEY (user_id) REFERENCES users (user_id),
  CONSTRAINT fk_bookings_show FOREIGN KEY (show_id) REFERENCES shows (show_id),
  CONSTRAINT chk_bookings_status CHECK (status IN ('CONFIRMED', 'CANCELLED'))
);
```

Listing 1: DDL of the BOOKINGS table

3.3 Normalization steps

- Unnormalised form: a paper ticket TICKET(booking_id, user details, movie details, screen, show_time, price, {row, seat_no, seat_type, seat_price}, total, status) contains a repeating group of seats.
- 1NF: the repeating group is removed so each seat is stored as its own row with atomic values, never as a list such as “A1,A2”.
- 2NF: attributes depending on only part of the key (booking_id, seat) are separated: booking details go to BOOKINGS, seat details to SEATS, and BOOKING_SEATS keeps only price, which depends on the whole key.
- 3NF: transitive dependencies such as booking_id → user_id → email and booking_id → show_id → movie_id → title are removed into USERS, SHOWS, MOVIES and SCREENS.
- BCNF: every determinant is a candidate key, for example email in USERS, (screen_id, show_time) in SHOWS and (screen_id, row_label, seat_no) in SEATS.

BOOKINGS.total_amount is a deliberate stored summary kept correct by a trigger, while seat availability is never stored but always computed from confirmed bookings.

4. Implementation

All business rules are inside the package movie_pkg and three triggers. The application only calls the package, so the same rules apply to every client.

4.1 Package

movie_pkg groups four functions and four procedures behind one specification. The constant c_max_seats defines the six-seat limit in one place.

```
CREATE OR REPLACE PACKAGE movie_pkg AS
  c_max_seats CONSTANT PLS_INTEGER := 6;

  FUNCTION seat_price(p_show_id NUMBER, p_seat_id NUMBER) RETURN NUMBER;
  FUNCTION available_seats(p_show_id NUMBER) RETURN NUMBER;
  FUNCTION refund_calc(p_amount NUMBER, p_show_time TIMESTAMP) RETURN NUMBER;
  FUNCTION seat_map(p_show_id NUMBER) RETURN SYS_REFCURSOR;

  PROCEDURE book_seats(p_user_id NUMBER, p_show_id NUMBER, p_seats VARCHAR2,
    p_booking_id OUT NUMBER);
  PROCEDURE cancel_booking(p_booking_id NUMBER);
  PROCEDURE show_report(p_show_id NUMBER);
  PROCEDURE daily_schedule(p_day DATE);
END movie_pkg;
```

Listing 2: Package specification of movie_pkg

4.2 Functions

seat_price returns the price of a seat for a show; premium seats (rows D and E) cost 1.5 times the base price.

```
FUNCTION seat_price(p_show_id NUMBER, p_seat_id NUMBER) RETURN NUMBER IS
  v_base shows.base_price%TYPE;
  v_type seats.seat_type%TYPE;
BEGIN
  SELECT base_price INTO v_base FROM shows WHERE show_id = p_show_id;
  SELECT seat_type INTO v_type FROM seats WHERE seat_id = p_seat_id;
  RETURN CASE v_type WHEN 'PREMIUM' THEN v_base * 1.5 ELSE v_base END;
END seat_price;
```

Listing 3: Function seat_price

available_seats counts the seats of the show's screen and subtracts the seats in confirmed bookings, so cancelled bookings free their seats automatically.

```
FUNCTION available_seats(p_show_id NUMBER) RETURN NUMBER IS
  v_total  NUMBER;
  v_booked NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_total
  FROM seats st JOIN shows sh ON st.screen_id = sh.screen_id
  WHERE sh.show_id = p_show_id;

  SELECT COUNT(*) INTO v_booked
  FROM booking_seats bs JOIN bookings b ON bs.booking_id = b.booking_id
  WHERE b.show_id = p_show_id AND b.status = 'CONFIRMED';

  RETURN v_total - v_booked;
END available_seats;
```

Listing 4: Function available_seats

refund_calc applies the refund policy: 100% more than two hours before the show, 50% within two hours and nothing after it starts.

```
FUNCTION refund_calc(p_amount NUMBER, p_show_time TIMESTAMP) RETURN NUMBER IS
  v_now TIMESTAMP := CAST(SYSDATE AS TIMESTAMP);
BEGIN
  IF p_show_time <= v_now THEN
    RETURN 0;
  ELSIF p_show_time - v_now > INTERVAL '2' HOUR THEN
    RETURN p_amount;
  ELSE
```

```
    RETURN ROUND(p_amount * 0.5, 2);
  END IF;
END refund_calc;
```

Listing 5: Function refund_calc

4.3 Procedures

book_seats validates the seat list, locks the show row with FOR UPDATE so concurrent bookings are handled one at a time, inserts the booking and its seats, logs it and commits. Any error rolls back the whole booking.

```
PROCEDURE book_seats(p_user_id NUMBER, p_show_id NUMBER, p_seats VARCHAR2,
                     p_booking_id OUT NUMBER) IS
  ...
BEGIN
  v_count := NVL(REGEXP_COUNT(p_seats, '[^,]+'), 0);
  IF v_count = 0 THEN
    RAISE_APPLICATION_ERROR(-20004, 'No seats selected');
  ELSIF v_count > c_max_seats THEN
    RAISE_APPLICATION_ERROR(-20009, 'Maximum ' || c_max_seats || ' seats per
    booking');
  END IF;
  SELECT screen_id, show_time INTO v_screen, v_time
  FROM shows WHERE show_id = p_show_id FOR UPDATE;      -- lock the show
  INSERT INTO bookings (user_id, show_id) VALUES (p_user_id, p_show_id)
  RETURNING booking_id INTO p_booking_id;
  FOR i IN 1..v_count LOOP
    ...                                                 -- validate seat label
    SELECT COUNT(*) INTO v_taken FROM booking_seats bs JOIN bookings b
      ON bs.booking_id = b.booking_id
    WHERE b.show_id = p_show_id AND b.status = 'CONFIRMED' AND bs.seat_id =
    v_seat;
    IF v_taken > 0 THEN
      RAISE_APPLICATION_ERROR(-20003, 'Seat ' || v_tok || ' already booked');
    END IF;
    INSERT INTO booking_seats (booking_id, seat_id) VALUES (p_booking_id, v_seat);
  END LOOP;
  INSERT INTO booking_log (booking_id, action, amount) VALUES (p_booking_id,
    'BOOKED', v_amt);
  COMMIT;
EXCEPTION
  WHEN OTHERS THEN ROLLBACK; RAISE;
END book_seats;
```

Listing 6: Procedure book_seats (key lines)

cancel_booking locks the booking, rejects double cancellation and cancellation after the show starts, and sets the status to CANCELLED. The refund is recorded by a trigger. The procedures show_report and daily_schedule are shown under Cursors.

```
PROCEDURE cancel_booking(p_booking_id NUMBER) IS
BEGIN
  SELECT b.status, s.show_time INTO v_status, v_time
  FROM bookings b JOIN shows s ON b.show_id = s.show_id
  WHERE b.booking_id = p_booking_id FOR UPDATE OF b.status;
  IF v_status = 'CANCELLED' THEN
    RAISE_APPLICATION_ERROR(-20006, 'Booking already cancelled');
  ELSIF v_time <= CAST(SYSDATE AS TIMESTAMP) THEN
    RAISE_APPLICATION_ERROR(-20008, 'Show has already started');
  END IF;
  UPDATE bookings SET status = 'CANCELLED' WHERE booking_id = p_booking_id;
  COMMIT;
```

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20007, 'Booking not found');
END cancel_booking;
```

Listing 7: Procedure cancel_booking (key lines)

4.4 Triggers

trg_seat_price is a BEFORE INSERT row trigger that sets each booked seat's price, so the caller cannot supply or alter it.

```
CREATE OR REPLACE TRIGGER trg_seat_price
BEFORE INSERT ON booking_seats
FOR EACH ROW
DECLARE
  v_show bookings.show_id%TYPE;
BEGIN
  SELECT show_id INTO v_show FROM bookings WHERE booking_id = :NEW.booking_id;
  :NEW.price := movie_pkg.seat_price(v_show, :NEW.seat_id);
END;
/
```

Listing 8: Trigger trg_seat_price

trg_booking_total is an AFTER INSERT row trigger that adds each seat's price to the booking total.

```
CREATE OR REPLACE TRIGGER trg_booking_total
AFTER INSERT ON booking_seats
FOR EACH ROW
BEGIN
  UPDATE bookings
  SET total_amount = total_amount + :NEW.price
  WHERE booking_id = :NEW.booking_id;
END;
/
```

Listing 9: Trigger trg_booking_total

trg_booking_cancel fires only when a booking changes from CONFIRMED to CANCELLED (WHEN clause) and logs the refund from refund_calc.

```
CREATE OR REPLACE TRIGGER trg_booking_cancel
AFTER UPDATE OF status ON bookings
FOR EACH ROW
WHEN (OLD.status = 'CONFIRMED' AND NEW.status = 'CANCELLED')
DECLARE
  v_time shows.show_time%TYPE;
BEGIN
  SELECT show_time INTO v_time FROM shows WHERE show_id = :NEW.show_id;
  INSERT INTO booking_log (booking_id, action, amount)
  VALUES (:NEW.booking_id, 'CANCELLED', movie_pkg.refund_calc(:NEW.total_amount,
    v_time));
END;
/
```

Listing 10: Trigger trg_booking_cancel

4.5 Cursors

Explicit cursor (show_report): c_bk is opened, fetched into a %ROWTYPE record until %NOTFOUND, counted with %ROWCOUNT and closed, to print the bookings, occupancy and revenue of a show.

```
CURSOR c_bk IS
  SELECT b.booking_id, u.name, b.total_amount, b.status, ... AS seat_list
  FROM bookings b JOIN users u ON b.user_id = u.user_id
  WHERE b.show_id = p_show_id ORDER BY b.booking_id;
v_rec c_bk%ROWTYPE;
...
OPEN c_bk;
LOOP
  FETCH c_bk INTO v_rec;
  EXIT WHEN c_bk%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE(RPAD(v_rec.booking_id, 6) || RPAD(v_rec.name, 16) || ...);
  IF v_rec.status = 'CONFIRMED' THEN v_rev := v_rev + v_rec.total_amount; END IF;
END LOOP;
DBMS_OUTPUT.PUT_LINE('Bookings: ' || c_bk%ROWCOUNT);
CLOSE c_bk;
```

Listing 11: Explicit cursor in procedure show_report

Parameterised cursor (daily_schedule): c_sh takes a date and is used in a cursor FOR loop to list that day's shows with free seats.

```
CURSOR c_sh(d DATE) IS
  SELECT s.show_id, m.title, sc.screen_name, s.show_time, s.base_price
  FROM shows s JOIN movies m ON s.movie_id = m.movie_id
  JOIN screens sc ON s.screen_id = sc.screen_id
  WHERE s.show_time >= d AND s.show_time < d + 1
  ORDER BY s.show_time, sc.screen_name;
...
FOR r IN c_sh(TRUNC(p_day)) LOOP
  DBMS_OUTPUT.PUT_LINE(TO_CHAR(r.show_time, 'HH24:MI') || '  ' || RPAD(r.title,
    16)
                       || ... || available_seats(r.show_id));
END LOOP;
```

Listing 12: Parameterised cursor in procedure daily_schedule

REF CURSOR (seat_map): the function opens a SYS_REFCURSOR over every seat of a show with its price and status and returns it to the application, which draws the seat map from it.

```
FUNCTION seat_map(p_show_id NUMBER) RETURN SYS_REFCURSOR IS
  rc SYS_REFCURSOR;
BEGIN
  OPEN rc FOR
    SELECT st.seat_id, st.row_label, st.seat_no, st.seat_type,
           movie_pkg.seat_price(p_show_id, st.seat_id) AS price,
           CASE WHEN EXISTS (SELECT 1
                             FROM booking_seats bs JOIN bookings b ON
    bs.booking_id = b.booking_id
                             WHERE b.show_id = p_show_id
                               AND b.status = 'CONFIRMED'
                               AND bs.seat_id = st.seat_id)
                THEN 'BOOKED' ELSE 'FREE' END AS status
    FROM seats st JOIN shows s ON st.screen_id = s.screen_id
    WHERE s.show_id = p_show_id
```

```
    ORDER BY st.row_label, st.seat_no;
  RETURN rc;
END seat_map;
```

Listing 13: Function seat_map returning a REF CURSOR

4.6 Exception handling

Table 3: Application error codes

Code	Raised by	Meaning
-20001	book_seats	Show has already started
-20002	book_seats	Invalid seat label, or seat not in this screen
-20003	book_seats	Seat already booked for this show
-20004	book_seats	No seats selected
-20005	book_seats	Show not found
-20006	cancel_booking	Booking already cancelled
-20007	cancel_booking	Booking not found
-20008	cancel_booking	Show has already started
-20009	book_seats	More than 6 seats in one booking

5. Front-End

The front-end is a desktop application. Its interface (HTML, CSS, JavaScript) calls a Python FastAPI server, which connects to Oracle through a python-oracledb connection pool and calls movie_pkg. Booking calls book_seats and receives the booking number through its OUT parameter; the seat map is read from the seat_map REF CURSOR; cancelling calls cancel_booking; and the Reports page shows show_report output captured from DBMS_OUTPUT. The database password stays on the server and every rule is enforced by PL/SQL.

```
booking_id = cur.var(int)
cur.callproc("movie_pkg.book_seats",
             [body.user_id, body.show_id, ",".join(body.seats), booking_id])
```

Listing 14: The application calling book_seats

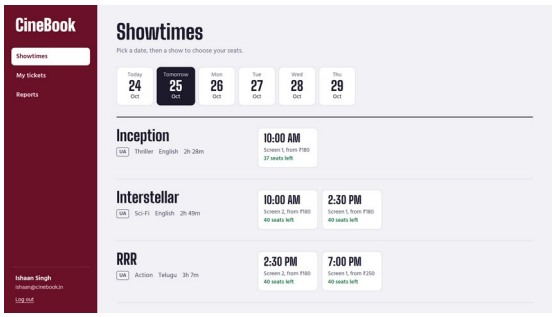


Figure 2: Showtimes with seats left from available_seats

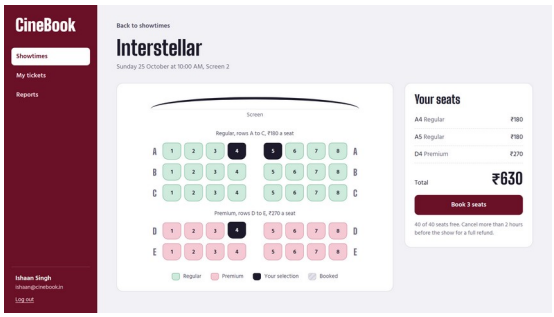


Figure 3: Seat map from the seat_map REF CURSOR

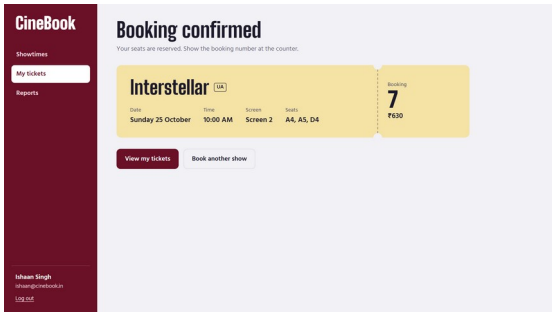


Figure 4: Booking confirmed by book_seats

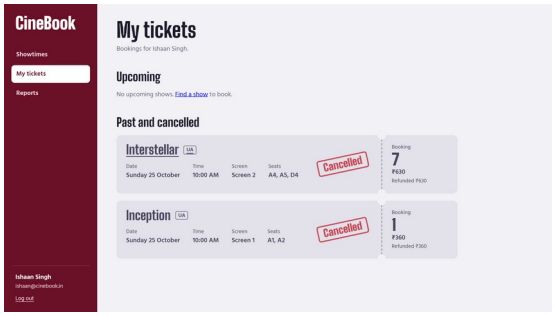


Figure 5: Cancelled booking with refund

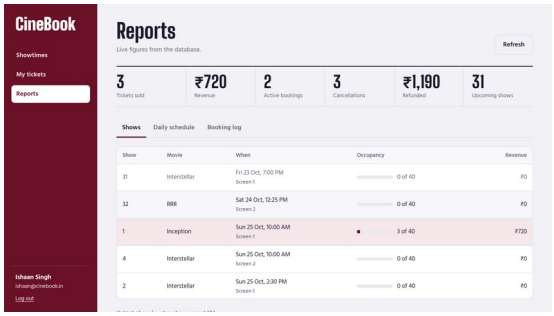


Figure 6: Reports with show_report output

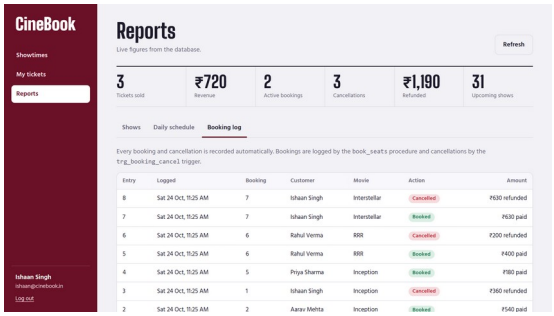


Figure 7: Booking log written by the procedure and trigger

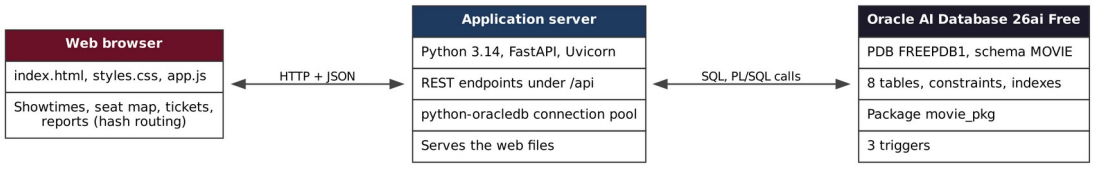


Figure 8: System architecture

6. Verification & Testing

6.1 Test cases

04_tests.sql runs 16 self-checking test cases on fresh data (reset.sql). Each prints PASS or FAIL; all 16 passed.

Table 4: Test cases and results

ID	Test	Expected result	Result
TC01	Free seats on a new show	40 of 40 free	PASS
TC02	Book regular seats A1, A2	Booking created, 38 seats left	PASS
TC03	Book premium seats D1, D2	₹270 per premium seat; totals ₹360 and ₹540	PASS
TC04	Book A3, A2 when A2 is taken	ORA-20003; A3 rolled back, 36 seats still free	PASS
TC05	Book seat Z9	ORA-20002 Invalid seat Z9	PASS
TC06	Book with an empty seat list	ORA-20004 No seats selected	PASS
TC07	Book 7 seats in one booking	ORA-20009 Maximum 6 seats per booking	PASS

ID	Test	Expected result	Result
TC08	Book a show that has started	ORA-20001 Show has already started	PASS
TC09	Cancel more than 2 hours before	Refund ₹360 (100%), A1 and A2 released	PASS
TC10	Cancel the same booking twice	ORA-20006 Booking already cancelled	PASS
TC11	Rebook released seat A1	New booking succeeds	PASS
TC12	Cancel within 2 hours of the show	Paid ₹400, refund ₹200 (50%)	PASS
TC13	Insert user with phone 12345	ORA-02290 CHK_USERS_PHONE violated	PASS
TC14	Fetch the seat map of show 1	40 seats returned, 3 booked	PASS
TC15	Report for show 1	Occupancy 3/40, revenue ₹720	PASS
TC16	Schedule for tomorrow	Schedule printed with free seats	PASS

6.2 Output screenshots

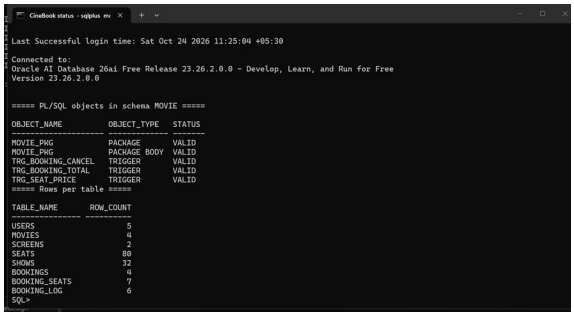


Figure 9: Package and triggers compiled VALID

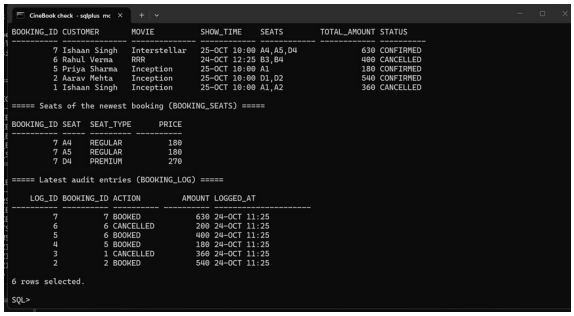


Figure 10: Booking 7 from the application, read in SQL*Plus

6.3 Validation of functionality

Booking 7 was made in the application (Figure 4) and immediately appeared in SQL*Plus with the same seats, trigger-set prices and a BOOKED log entry (Figure 10), proving that the front-end stores its data in Oracle. Cancelling it in the application changed its status to CANCELLED and added the refund entry shown in the booking log.

7. Results & Discussion

7.1 Achievements

- A BCNF schema of eight tables with 27 named constraints.
- A package of four functions and four procedures, three triggers and all cursor types.
- Double-booking prevention, automatic pricing, refunds and audit logging.
- 16 of 16 test cases passing and a working desktop front-end.

7.2 Challenges faced

- Concurrent bookings of the same seat, solved by locking the show row with SELECT ... FOR UPDATE.
- Keeping the stored booking total consistent, solved by letting only a trigger change it.
- Testing time-based refund rules, solved by creating a test show one hour ahead.
- Showing DBMS_OUTPUT in the application, solved by reading it back with DBMS_OUTPUT.GET_LINE.

7.3 Possible improvements

Password login with customer and manager roles, temporary seat holds, online payment, QR e-tickets and support for multiple cinemas.

8. Conclusion

CineBook shows that consistency, pricing and record keeping are best enforced inside the database. The project gave practical experience in schema design and normalisation, PL/SQL packages, functions, procedures, triggers and cursors, exception handling, transactions and locking, and connecting an application to Oracle. Future scope includes authentication with roles, seat holds and online payment.

References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[3] Oracle Corporation, “Database PL/SQL Language Reference, 23ai,” Oracle Help Center. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/23/lnpls/ (accessed Oct. 23, 2026).

[4] Oracle Corporation, “python-oracledb documentation.” [Online]. Available: https://python-oracledb.readthedocs.io/ (accessed Oct. 23, 2026).

Appendices

Appendix A: Test run output

```
TC01 PASS: 40 of 40 seats free
TC02 PASS: booking 1 created, 38 seats left
TC03 PASS: regular total Rs 360, premium total Rs 540
TC04 PASS: ORA-20003: Seat A2 already booked (A3 rolled back, 36 seats still free)
TC05 PASS: ORA-20002: Invalid seat Z9
TC06 PASS: ORA-20004: No seats selected
TC07 PASS: ORA-20009: Maximum 6 seats per booking
TC08 PASS: ORA-20001: Show has already started
TC09 PASS: refund Rs 360, seats A1 and A2 released
TC10 PASS: ORA-20006: Booking already cancelled
TC11 PASS: A1 rebooked as booking 5
TC12 PASS: paid Rs 400, refund Rs 200
TC13 PASS: ORA-02290: check constraint (MOVIE.CHK_USERS_PHONE) violated
TC14 PASS: 40 seats returned, 3 booked (A1, D1, D2)
TC15 PASS: report generated, expected occupancy 3/40 and revenue Rs 720
TC16 PASS: schedule generated
16 of 16 test cases passed
```

Appendix B: Project files

Table 5: Project files

File	Contents
sql/01_schema.sql, 02_data.sql	Tables, constraints, indexes and sample data
sql/03_plsql.sql	Package movie_pkg and the three triggers
sql/04_tests.sql, test_results.txt	Test cases and their output
sql/check.sql	Query used to show stored bookings
api/main.py, api/desktop.py	Application server and desktop launcher
web/	Front-end